

Оглавление

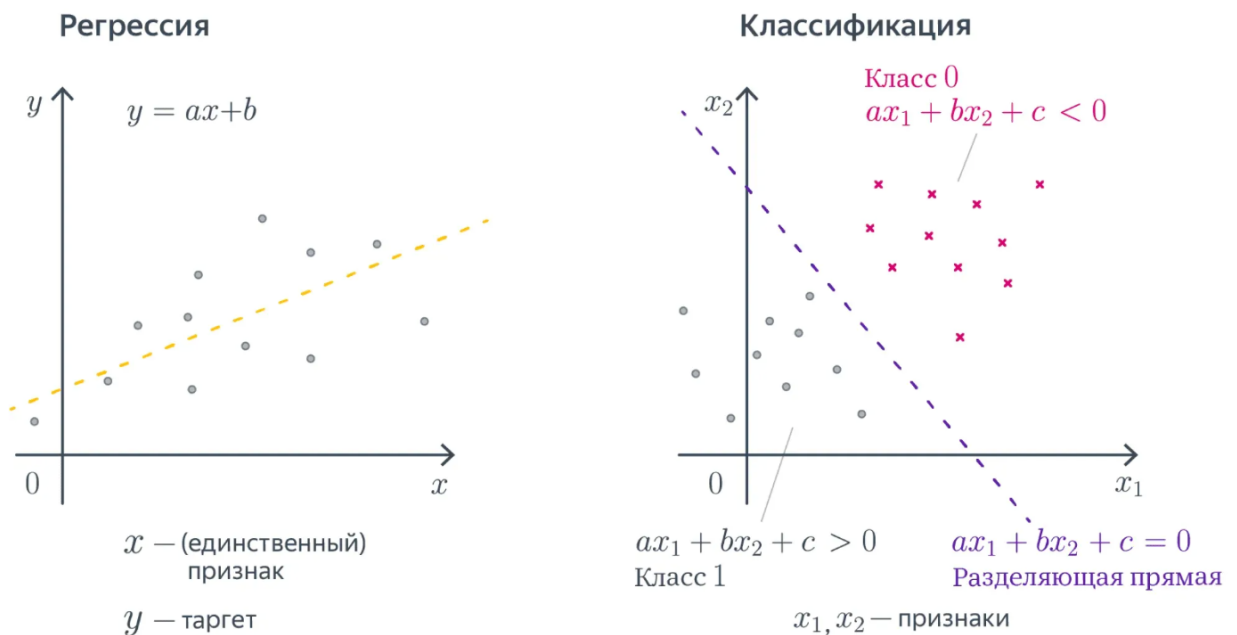
<i>Обучение с учителем (Supervised Learning)</i>	<i>2</i>
<i>Классификация</i>	<i>4</i>
<i>Метрики качества</i>	<i>5</i>
Матрица ошибок (Confusion Matrix)	5
Accuracy	6
Precision	7
Recall	8
F1-Score	8
AUC-ROC	9
Метрики, которые используют уверенность	12
Калибровка модели	12
<i>Регрессия</i>	<i>13</i>
Типы задач регрессии	13
Процесс обучения модели регрессии	13
<i>Метрики оценки</i>	<i>14</i>
Функция потерь (Loss Function)	15
Функция стоимости (Cost Function)	15
Функция награды (Reward Function)	16
<i>Градиентный спуск</i>	<i>16</i>

Обучение с учителем (Supervised Learning)

Обучение с учителем предполагает, что для каждого примера из обучающего набора данных известен правильный ответ (метка). Целью является построение модели, которая научится предсказывать метки для новых, ранее не виденных данных.

Классификация (KNN, Naive Bayes, SVM, Decision Tree, Logistic Regression): y - *дискретная метка класса* (например, распознавание изображений, пол человека, марка автомобиля, спам/не спам).

Регрессия (Liner Regression, Polynomial Regression, Ridge/Lass Regression): y - *непрерывная переменная* (например, предсказание цены дома, курс рубля).



Процесс обучения:

- 1) Выбор модели: это может быть линейная модель, дерево решений, нейронная сеть и т.д.
- 2) Обучение модели: на основе обучающих данных (фичи и метки) модель настраивает свои параметры так, чтобы минимизировать ошибку между предсказанными и реальными метками.
- 3) Валидация и тестирование: проверка качества модели на отдельном наборе данных для оценки ее способности к обобщению на новых данных.

В линейную модель можно подать только численные признаки, так что категориальную фичу придётся как-то закодировать.

	pet_type	color	weight
0	parrot	red	0,600
1	cat	black	3,000
2	hamster	brown	0,800
3	cat	gray	5,000
4	dog	black	7,000

В этом датасете два категориальных признака — pet_type и color. Каждый принимает по четыре различных значения. Самый простой способ — использовать one-hot-кодирование (англ. one-hot encoding).

One-hot-кодирование — это наивный способ работы с категориальными признаками, и для более сложных фич или фич с большим количеством значений оно плохо подходит.

Классификация

Цель - предсказание категориальных меток для данных. В этих задачах, данные или объекты необходимо классифицировать по определённым классам на основе их характеристик или признаков.

Основные понятия

Классы – predetermined группы или категории, к которым должны быть отнесены примеры данных. Например, в задаче по определению спама классы могут быть "спам" и "не спам".

Признаки (фичи) – характеристики или атрибуты данных, на основе которых модель обучается делать предсказания. Например, в текстовых данных это могут быть частоты встречаемости слов.

Метки – конкретные значения классов, присвоенные каждому объекту тренировочных данных. В процессе обучения использование меток позволяет модели "учиться" правильной классификации.

Типы задач классификации

1. Бинарная классификация

Когда имеются всего два класса. Пример – определение, является ли электронное письмо спамом.

2. Многоклассовая классификация (мультиклассовая классификация)

Когда объекты классифицируются на более чем два класса. Пример – классификация новостных статей по тематикам.

3. Многокатегорийная классификация (мультилейбл классификация)

Когда объект может одновременно принадлежать к нескольким классам. Пример – классификация изображения, на котором изображены несколько объектов.

Процесс обучения модели классификации

- 1) Предобработка данных включает в себя нормализацию признаков, заполнение пропущенных значений, кодирование категориальных переменных и т.д.
- 2) Выбор модели. В зависимости от задачи и данных это может быть **логистическая регрессия, дерево решений, случайный лес, градиентный бустинг, нейронная сеть и т.д.**
- 3) Обучение модели. В процессе обучения модель "изучает" взаимосвязь между признаками и метками на тренировочном наборе данных.

- 4) Оценка модели. Проверка качества классификации модели на валидационном и тестовом наборах данных. Используются метрики, как **точность (accuracy)**, **точность (precision)**, **полнота (recall)**, **F-мера** и другие.

Метрики оценки

Точность (Accuracy): Общая доля правильно классифицированных случаев из всех случаев.

Точность (Precision): Доля правильно классифицированных положительных случаев из всех случаев, классифицированных как положительные.

Полнота (Recall): Доля правильно классифицированных положительных случаев из всех реальных положительных случаев.

F-мера (F1-Score): Среднее гармоническое точности и полноты. Используется, когда требуется сбалансировать между точностью и полнотой.

Метрики качества в заданных классификациях

В задачах классификации в машинном обучении важно не только настроить модель для предсказания классов объектов, но и уметь оценивать качество её работы. Существует несколько ключевых метрик, позволяющих объективно оценить эффективность классификатора.

Матрица ошибок (Confusion Matrix)

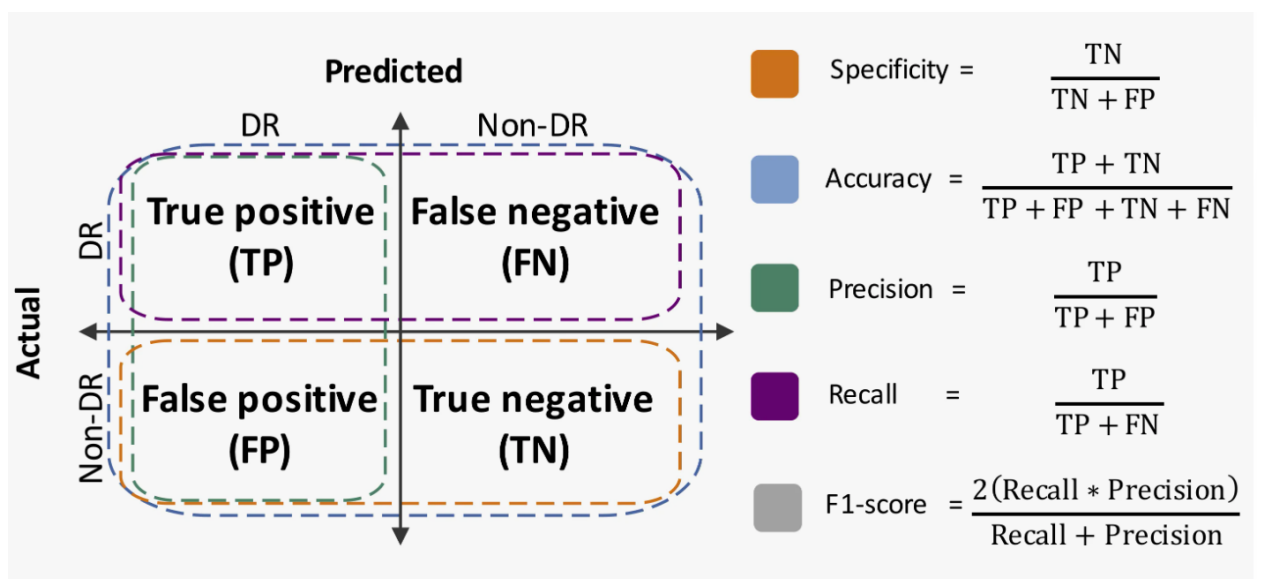
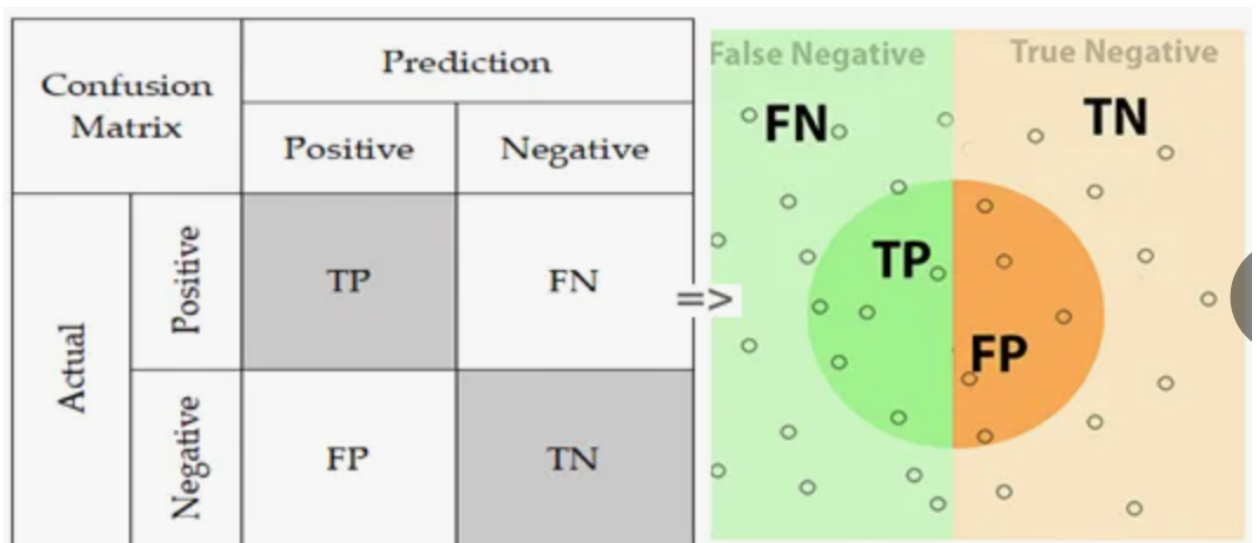
В машинном обучении, особенно в задачах классификации, важно не просто знать, сколько ошибок делает модель, но и различать качество этих ошибок. Для этого существует матрица ошибок (Confusion Matrix), которая делит предсказания модели на 4 категории. Давайте представим, что мы обучаем модель для будущего беспилотного автомобиля. Она должна определить наличие светофора на фото.

1. True Positive (TP). Верный положительный ответ. Модель определила, что светофор на фото присутствует. И на самом деле он присутствует.
2. False Positive (FP). Ложный положительный ответ. Модель определила, что светофор есть, но на самом деле его нет.
3. True Negative (TN). Верный отрицательный ответ. Модель сказала, что светофора нет. И на самом деле его нет.
4. False Negative (FN). Ложный отрицательный ответ. Модель сказала, что светофора нет, но на самом деле он есть.

Такая классификация помогает:

- Считать метрики качества модели

- Понимать, где ошибается модель
- Выбирать оптимальный порог классификации



Accuracy

Accuracy — это одна из ключевых метрик для оценки качества классификационных моделей. Она показывает долю правильных предсказаний модели относительно общего числа предсказаний.

Кол-во верных предсказаний / Общее кол-во предсказаний * 100%

Пример:

Модель говорит, что светофор присутствует или его нет. Из 100 тестовых фото:

Верно предсказано: 90 случаев (85, где светофор есть + 5, где он отсутствует).

Ошибки: 10 случаев.

Данная метрика подходит только в тех случаях, в которых классы сбалансированы. Т.е., у нас есть 50 фото со светофором и 50 без него. Кроме того, если важна только общая доля правильных ответов (без детализации по типам ошибок) и нам не важно, ошиблась модель, отвечая “да”, когда светофора не было или “нет”, когда он был.

Если же датасет не сбалансирован, например, у нас 85 фото со светофором и 15 без него, то данная метрика не имеет смысла. Модель, которая всегда предсказывает “да”, получит Accuracy в 85%, но результат будет мягко говоря неточный, так как неизвестно каким бы он был при противоположном соотношении. Данная метрика очень грубая, не различает False Positive (ложные срабатывания) от False Negative (пропуск цели) и поэтому, используется в редких случаях.

Precision (Точность/Прецизионность)

Precision — это метрика, которая показывает, какая доля положительных предсказаний модели была верной. Она показывает, сколько раз была модель права из всех случаев, когда она сказала “да”.

Имеет следующую формулу: $TP / TP + FP$

Представим, что модель определила светофоры на 80 фото. Из них на 8 фото модель разглядела светофор там, где его нет (FP). Ошиблась.

$Precision = 72 / (72 + 8) = 0.9$ (90%).

Метрика критически важна, когда ложные срабатывания (FP) дорого обходятся.

Например, в спам-филтре ошибочно поместить важное письмо в спам (FP) хуже, чем пропустить спам (FN). Еще пример — системы безопасности, в которых ложная тревога (FP) тратит ресурсы большие ресурсы.

И пример, когда Precision не подходит, — поиск тяжелых заболеваний по медицинским снимкам. Если заболевание редкое, то датасет будет несбалансированным. Больных, к примеру, 1%, а здоровых — 99. Если модель скажет “болен” только для 1 пациента из 1000, то Precision будет равен 100% ($1 / (1 + 0) = 1$). Модель сказала “болен” 1 раз и 1 раз она была права. Но при этом 9 больных умрут (10 больных из 1000 здоровых). Модель не считает их больными, а они таковыми являются (FN)

Recall (полнота)

Recall (или чувствительность) — это метрика, которая показывает, какую долю реальных положительных случаев модель смогла правильно идентифицировать. Показывает, из всех реальных "да", сколько модель нашла.

Рассчитывается по формуле: $TP / (TP + FN)$

Например, набор данных состоит из 100 фото. Из них на 60 присутствует светофор, а на 40 его нет. Модель из 60 фото со светофорами отметила только 52 светофора (8 раз модель сказала "нет", а на самом деле ответ был "да").

Рассчитаем метрику для этого случая:

$$\text{Recall} = 52 / (52 + 8) = 0.87 \text{ (87\%)}$$

Recall необходим, когда пропуск положительных случаев (FN) недопустим. Например, случаи в медицинской диагностике, при которых пропустить болезнь (FN) опаснее, чем создать ложную тревогу (FP) и назначить лечение по ошибке. При поиске мошенничества, когда невыявление реальной атаки (FN) принесет финансовые потери. В поисковых системах, когда пропуск релевантного документа (FN) может снизить качество выдачи.

В то же время, метрика не учитывает FP (модель пометила "светофор" на фото, а его там нет). Поэтому там, где она FP критично, данная метрика используется в паре с Precision и другими метриками.

F1-Score

Если важно учитывать и FP, и FN, используют F1-Score — среднее гармоническое Precision и Recall.

Формула: $2 * ((\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}))$

Или $(2 * TP) / (2 * TP + FP + FN)$

Из определения следует, что данная метрика используется там, где важно учитывать как ложные срабатывания (FP), так и пропуски (FN) для задач с несбалансированными классами.

Пример расчета:

$$\text{Precision} = 0.8, \text{Recall} = 0.8$$

$$2 * ((0.8 * 0.8) / (0.8 + 0.8)) == 0.8$$

Данная модель одинаково хороша в точности и полноте. Данный вывод делается на основе того, что F1 стремится к 1.

Пример с дисбалансом в сторону Precision

$$2 * ((0.8 * 0.4) / (0.8 + 0.4)) == 0.53$$

Из примеров видно, что если одна из метрик ближе к 0, то и F1 стремится к нулю. Т.е., *идеальная модель с точки зрения данной метрики – это модель, при которой F1 равна 1.*

AUC-ROC (Area Under the ROC Curve) — площадь под ROC-кривой

AUC-ROC — это метрика, которая оценивает качество бинарной классификации независимо от порога классификации. Она показывает, насколько модель способна отличать положительные классы от отрицательных. Чем выше AUC-ROC (максимум 1), тем лучше модель. То есть это метрика, которая показывает, насколько хорошо модель отличает один класс от другого (фото со светофорами от фото без светофоров). **Чем ближе AUC-ROC к 1, тем лучше модель.** Если AUC-ROC = 0.5, модель работает как случайное угадывание.

Сначала модель предсказывает вероятность. Например, для каждого фото модель выдает число от 0 до 1. При этом, 1 будет означать, что модель на 100% уверена в том, что на фото есть светофор, а 0 – что его нет.

Постепенно начинаем изменять порог уверенности. Сначала мы говорим, что если уверенность > 0.9, то светофор есть. Затем, что если > 0.8. И так далее до 0. На каждом шаге мы записываем результат.

Результат состоит из следующих категорий

True Positive Rate (TPR). Показывает, сколько фото со светофорами были правильно помечены. Вычисляется по формуле:

$$TP / (TP + FN)$$

False Positive Rate (FPR). Отражает, сколько фото были помечены по ошибке. Формула

$$FP / (FP + TN)$$

Итак, после того, как мы изменили порог уверенности и произвели замеры TPR и FPR, строим ROC-кривую, где по оси X находится FPR, а по оси Y – TPR.

Идеальная модель сразу даёт TPR=1 (все светофоры найдены) и FPR=0 (нет ложных срабатываний). Плохая же модель идёт по диагонали. Т.е., её предсказания больше похожи на рулетку.

Разберем примеры без лишних графиков и усложнений.

Представим, что всем фото со светофорами модель присвоила вероятность, близкую к 1 (например, 0.99, 0.95, 0.98). А фото без светофоров получили вероятность, близкую к 0 (например, 0.01, 0.02, 0.05).

Далее, сравниваем попарно.

1. $0.99 > 0.01$
2. $0.99 > 0.02$
3. $0.99 > 0.05$
4. $0.95 > 0.01$
5. $0.95 > 0.02$
6. $0.95 > 0.05$
7. $0.98 > 0.01$
8. $0.98 > 0.02$
9. $0.98 > 0.05$

Все сравнения верны. Т.е., в 100% случаев. Это означает, что $AUC-ROC = 1.0$.

Если же представить, что один светофор был определен по ошибке (например, 0.01, 0.02, 0.7), а один остался незамеченным (например, 0.99, 0.95, 0.6), то расклад уже будет другим.

1. $0.99 > 0.01$
2. $0.99 > 0.02$
3. $0.99 > 0.7$
4. $0.95 > 0.01$
5. $0.95 > 0.02$
6. $0.95 > 0.7$
7. $0.6 > 0.01$
8. $0.6 > 0.02$
9. $0.6 < 0.7$

Теперь $AUC-ROC < 1.0$, так как только 8 из 9 сравнений имеют положительный результат (88.89%).

В отличие от Accuracy, AUC-ROC устойчива к дисбалансу классов. Поэтому, ее рекомендуется использовать при несбалансированности данных. Также, метрика подходит там, где важны вероятности. Например, ранжирование заемщиков по риску.

В то же время, AUC-ROC не очень хорошо подходит для многоклассовой классификации.

Уверенность модели (model confidence)

Уверенность модели — это числовое выражение того, насколько уверенно модель делает свое предсказание. Обычно она представляется в виде вероятностей, выдаваемых на выходе softmax (в классификации) или сигмоидной функции (в бинарной классификации или multilabel-задачах).

Оценка уверенности модели особенно важна в задачах медицины, автономного вождения, финансов, информационного поиска — там, где ошибка может дорого стоить; в активном обучении для выбора наиболее неуверенных примеров; в калибровке моделей и постобработке предсказаний; - в системах с human-in-the-loop, чтобы человек проверял предсказания, в которых модель не уверена.

Как измеряется уверенность

Типичный способ — взять максимальное значение вероятностей, выдаваемых моделью. Для классификации:

$$\text{Confidence} = \max(p_1, p_2, \dots, p_k)$$

где p_i — вероятность, присвоенная модели классу i ,

k — общее число классов.

Пример:

Пусть softmax на выходе дал:

[0.10, 0.05, 0.02, 0.78, 0.05]

Значит, модель предсказывает класс №4 с уверенностью 0.78.

Как интерпретировать:

- Высокая уверенность + правильный ответ — модель работает хорошо.
- Высокая уверенность + ошибка — опасный случай, возможна переуверенность.
- Низкая уверенность + ошибка — допустимо, модель "понимает", что не уверена.
- Низкая уверенность + правильный ответ — может свидетельствовать о неустойчивости модели.

Метрики, которые используют уверенность:

Brier Score — оценивает точность предсказанных вероятностей.

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Где p_i — предсказанная вероятность, $y_i \in \{0,1\}$ — истинная метка.

Log Loss (Cross-Entropy) — штрафует за уверенные, но неправильные предсказания:

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k y_{ij} \log(p_{ij})$$

Плюсы

- Позволяет не только предсказывать, но и понимать, насколько можно доверять модели.
- Необходима в риск-ориентированных приложениях.
- Используется при построении доверительных интервалов и калибровке.

Минусы

- Уверенность $\neq$ точность. Модель может быть уверенной, но ошибаться.
- Модели часто переуверены (особенно нейросети без калибровки).
- Требуется дополнительный анализ или калибровка, чтобы использовать уверенность корректно.

Калибровка модели (в связке с уверенностью)

Модель может предсказывать 0.95, но при этом быть права только в 70% таких случаев. Это значит — модель не откалибрована. Типичные методы калибровки:

- Platt Scaling (для бинарной классификации)
- Temperature Scaling (для softmax)

- Isotonic Regression (непараметрический способ)

Регрессия

Регрессия заключается в нахождении зависимости между набором независимых переменных (признаками) и зависимой переменной (целевым признаком). Задача регрессии сводится к построению модели регрессии, которая на основе предоставляемых данных способна с максимальной точностью предсказывать значение зависимой переменной.

Типы задач регрессии

Линейная регрессия

Предполагает линейную связь между входными переменными и целевой переменной. В простейшем случае одной переменной это может быть уравнение вида $y = mx + b$, где y – целевая переменная, x – входная переменная, m и b – параметры модели, которые необходимо определить.

Множественная регрессия

Расширение линейной регрессии для случаев, когда для предсказания используется несколько входных переменных.

Полиномиальная регрессия

В случае, когда отношение между входными переменными и выходной переменной лучше описывается полиномом, высшей степени.

Регрессия с регуляризацией

Применяется для избежания переобучения модели путем добавления штрафа за слишком высокие значения параметров модели (например, Lasso или Ridge регрессия).

Процесс обучения модели регрессии

- 1) Сбор и подготовка данных: Очистка, нормализация, а также выбор и создание соответствующих признаков.
- 2) Выбор модели: Определение, какая модель регрессии лучше всего подходит для конкретной задачи.
- 3) Обучение модели: Использование тренировочного набора данных для подгонки модели, т.е. определения её параметров.
- 4) Оценка модели: Проверка качества модели с использованием тестового набора данных и соответствующих метрик (например, среднеквадратичная ошибка (MSE), коэффициент детерминации (R^2)).

Метрики оценки задач регрессии:

- Среднеквадратичная ошибка (MSE)

Среднее квадратов ошибок между предсказанными и истинными значениями. Чем ниже значение MSE, тем лучше модель. Среднеквадратичная ошибка (MSE) является стандартной метрикой для измерения качества модели машинного обучения при задачах регрессии. Она вычисляется как среднее арифметическое квадратов разностей между предсказанными и истинными значениями.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

где:

- n — количество точек данных,
- y_i — истинное значение для i -ого примера,
- $\hat{y}_i$ — предсказанное значение для i -ого примера.

Главный плюс MSE — простота и интерпретируемость. Чем меньше значение, тем ближе предсказания к реальности. А за счёт квадрата ошибок, она жёстко наказывает большие отклонения, что важно в чувствительных к точности задачах, например, в прогнозировании финансов.

Метрика плохо переносит выбросы. Одна крупная ошибка может сильно исказить общий результат. Поэтому, если в данных возможны аномалии, стоит рассмотреть альтернативы вроде MAE (Mean Absolute Error).

- Коэффициент детерминации R^2

Коэффициент детерминации R^2 часто используется для оценки качества модели линейной регрессии. R^2 показывает, какая доля вариативности зависимой переменной объяснима независимыми переменными модели.

R^2 может принимать значения от $-\infty$ до 1. Значение 1 указывает на идеальное предсказание, значение 0 означает, что модель не лучше, чем простое предсказание среднего значения целевой переменной. Отрицательное значение R^2 может указывать на то, что модель очень плоха.

- Средняя абсолютная ошибка (MAE)

Средняя абсолютная ошибка (MAE) измеряет среднее значение абсолютных разностей между предсказанными и истинными значениями. Это позволяет оценить ошибки в тех же единицах измерения, что и переменная.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

где:

- $|y_i - \hat{y}_i|$ — абсолютная разность между истинным значением и предсказанием для i -ого примера.

MAE менее чувствительна к выбросам по сравнению с MSE, так как ошибки не возводятся в квадрат, обеспечивая более стабильную и последовательную оценку ошибок работы модели

Критерии оптимизации

В контексте машинного обучения, критерии оптимизации — это функции, которые должны быть минимизированы или максимизированы для достижения наилучшего возможного предсказательного качества модели. Эти критерии играют ключевую роль в процессе обучения модели, позволяя ей "учиться" на основе данных.

Функция потерь (Loss Function)

Функция потерь измеряет, насколько предсказания модели отличаются от фактических значений. Целью обучения является минимизация этой функции, что соответствует увеличению точности модели. Существует множество функций потерь, и выбор зависит от типа задачи (регрессия, классификация и т.д.).

- Среднеквадратичная ошибка (MSE): широко используется для задач регрессии.
- Перекрёстная энтропия (Cross-entropy): часто применяется в задачах классификации.

Функция стоимости (Cost Function)

Функция стоимости — это агрегированное представление потерь по всему набору данных. В контексте обучения с учителем, функция стоимости часто совпадает с функцией потерь, однако, она также может включать дополнительные термины, такие как регуляризация, для предотвращения переобучения.

Регуляризация (L1, L2): добавляется к функции стоимости для контроля сложности модели. L1-регуляризация (Лассо) определяется как сумма абсолютных значений коэффициентов модели, а L2-регуляризация (Ридж) — как сумма квадратов коэффициентов. Обе помогают предотвратить переобучение, "штрафуя" модели за слишком большие веса.

Функция награды (Reward Function)

Используется преимущественно в обучении с подкреплением, где модель "учится" на основе награды, получаемой в результате её действий. Целью является максимизация общей награды.

Способы оптимизации функций (градиентный спуск)

Для нахождения оптимальных параметров модели, которые минимизируют функцию потерь или стоимости, используются различные алгоритмы оптимизации (но мы будем рассматривать только один, остальные - более усовершенствованные градиентные спуски). Градиентный спуск — это как спуск с горы в темноте, имея только фонарик. Представь, что ты стоишь на вершине горы и твоя задача — спуститься в самую низкую точку долины, но темно, и ты видишь только небольшой круг от света фонарика. Ты не можешь увидеть всю долину сразу, чтобы выбрать лучший путь, поэтому ты решаешь двигаться шаг за шагом, выбирая направление, в котором земля уходит вниз больше всего.

В машинном обучении эта "гора" представляет собой функцию потерь, которую нужно минимизировать, или другими словами, задачу найти наилучшие параметры для модели, чтобы она работала как можно лучше. "Высота" в данной точке на "горе" показывает, насколько плохо в данный момент работает модель. Цель — спуститься к самой низкой точке, то есть найти параметры модели, при которых значение функции потерь будет минимальным (модель будет работать как можно лучше).

Фонарик в этой аналогии — это градиент функции потерь, который показывает направление крутейшего спуска из текущей точки. "Шагаем" мы, обновляя параметры модели в направлении, противоположном градиенту, потому что мы хотим идти вниз, а градиент указывает вверх, к повышению функции потерь.

Пример

Предположим, у нас есть набор данных с показаниями термометра за месяц и мы хотим предсказывать температуру на следующий день, используя простую линейную модель типа "температура завтра = вес × температура сегодня + смещение". Параметрами модели являются "вес" и "смещение".

Сначала мы инициализируем наши параметры "наугад" и считаем, насколько хорошо наша модель предсказывает температуру (это наша высокая точка на "горе"). Затем, используя градиентный спуск, постепенно адаптируем "вес" и "смещение" таким образом, чтобы наши предсказания становились всё ближе к реальным значениям (спускаемся ниже на "горе"). На каждом шаге мы проверяем градиент, чтобы узнать, в каком направлении нам следует корректировать наши параметры, чтобы сделать предсказание нашей модели ближе к действительности.

Градиентный спуск подскажет нам, как нужно изменить "вес" и "смещение", чтобы наша модель с каждым шагом становилась только точнее, пока мы не достигнем точки, где

модель предсказывает температуру с наименьшим возможным отклонением от реальных значений.

Итак, градиентный спуск — это оптимизационный алгоритм, используемый в машинном обучении для минимизации функции потерь, то есть для нахождения параметров модели, которые делают модель как можно более точной. Градиентный спуск основан на наблюдении, что если многомерная функция потерь $J(\theta)$ равномерно увеличивается или уменьшается в направлении градиента, то, двигаясь в противоположном градиенту направлении, можно приблизиться к минимуму функции.

Основное Уравнение Основное уравнение градиентного спуска для обновления параметров модели представляется как:

$$\theta := \theta - \eta \nabla_{\theta} J(\theta)$$

где:

- θ — параметры модели (например, веса),
- η — скорость обучения, которая определяет размер шага на каждой итерации,
- $\nabla_{\theta} J(\theta)$ — градиент функции потерь по отношению к параметрам.

Типы Градиентного Спуска:

- **Пакетный Градиентный Спуск**

При пакетном градиентном спуске для обновления параметров θ используется весь обучающий набор данных. Это обеспечивает стабильный путь к минимуму, но может быть вычислительно дорогостоящим при больших объемах данных.

- **Стохастический Градиентный Спуск (SGD)**

Стохастический градиентный спуск обновляет параметры θ по одному примеру за раз. Это может привести к быстрым, но более неравномерным обновлениям, что иногда позволяет быстрее найти достаточно хорошее решение.

- **Мини-пакетный Градиентный Спуск**

Мини-пакетный градиентный спуск является компромиссом между пакетным и стохастическим подходами, обновляя θ не на основе всего набора данных, а используя мини-пакеты фиксированного размера. Это способствует более эффективному использованию вычислительных ресурсов и дает возможность использовать оптимизации из области матричных вычислений.

Проблемы градиентного спуска

Градиентный спуск может "застрять" в локальном минимуме, который не является глобальным минимумом функции потерь.

Не существует универсального способа выбрать оптимальную скорость обучения.

В высокоразмерных пространствах седловые точки (где градиент равен нулю, но это не минимум) могут создавать проблемы для градиентного спуска.